
네트워크 게임 프로그래밍 프로젝트 Progress Report



2022182035 임채은(팀장)

2022180039 최미나

2022184031 임수영

목차

1. 애플리케이션 기획	3
A. 게임 소개	3
B. 수정 및 추가할 내용	3
C. 조작 Key	3
D. Monster와 Item	3
E. 스테이지	4
2. High-level 디자인	6
A. 플로우 차트	6
B. 서버와 클라이언트 흐름	7
3. Low-level 디자인	9
A. 패킷	9
B. 서버	11
C. 클라이언트	14 16
D. 멀티 스레드 함수	14 18
4. 팀원 간 역할 분담	15 19
5. 개발 환경	16 22
6. 개발 일정	17 22
7. 오류 및 해결 과정	17 28

1. 애플리케이션 기획

A. 게임 소개

이름	대초원의 모험
작업자	최미나, 임수영 (윈도우 프로그래밍)
장르	2D 탐류 아케이드 게임
참고한 게임	스타듀밸리 대초원 왕의 모험

B. 수정 및 추가할 내용

- GameNetwork 클래스 추가

C. 조작 Key

WASD	상하좌우 이동
방향키	총알 발사 방향
Space Bar	아이템 사용
Shift	폭탄 발로 차기

D. Monster와 Item

[Monster]

- 상하좌우 4방향에서 랜덤하게 생성
- 처치 시 효과가 나오고 확률적으로 아이템을 떨어뜨림
- 플레이어와 충돌시 플레이어의 목숨 -1 후 플레이어가 화면 중앙에서 부활함
- 몬스터 종류 및 특징

종류	특징	사망 조건
일반 몬스터	X	총알 1번 맞음
부활 몬스터	총알을 한 번 맞으면 기절하고 부활함	부활 후 총알 1번 맞음
탱커 몬스터	데미지가 큼	총알 5번 맞음

설치형 몬스터	일정 시간이 지나면 장애물로 변함	장애물 변신 전 1번, 장애물 변신 후 3번 맞음
폭탄 몬스터	시간 간격을 두고 폭탄 설치, 설치 시 정지함	총알 1번 맞음

[Item]

- 몬스터 처치 시 얻는 아이템

아이템	능력(일시)
캐릭터 얼굴	목숨 +1
탄창	공속 증가
벼락	게임 화면 내 모든 몬스터 처치
물레바퀴	모든 방향으로 총알 발사
커피	이동 속도 증가
총(샷건)	한 방향으로 총알 3개씩 발사
시계	몬스터 정지 시키기

E. 스테이지

- 총 4개의 스테이지
- 몬스터를 다 잡지 못하면 다음 스테이지로 넘어갈 수 없음
- 스테이지마다 각각 다른 장애물 존재
- 장애물: 나무 장애물, 설치형 몬스터, 굴러다니는 선인장(플레이어와 충돌 시 플레이어 목숨 -1)

스테이지 1



스테이지 2



스테이지 3

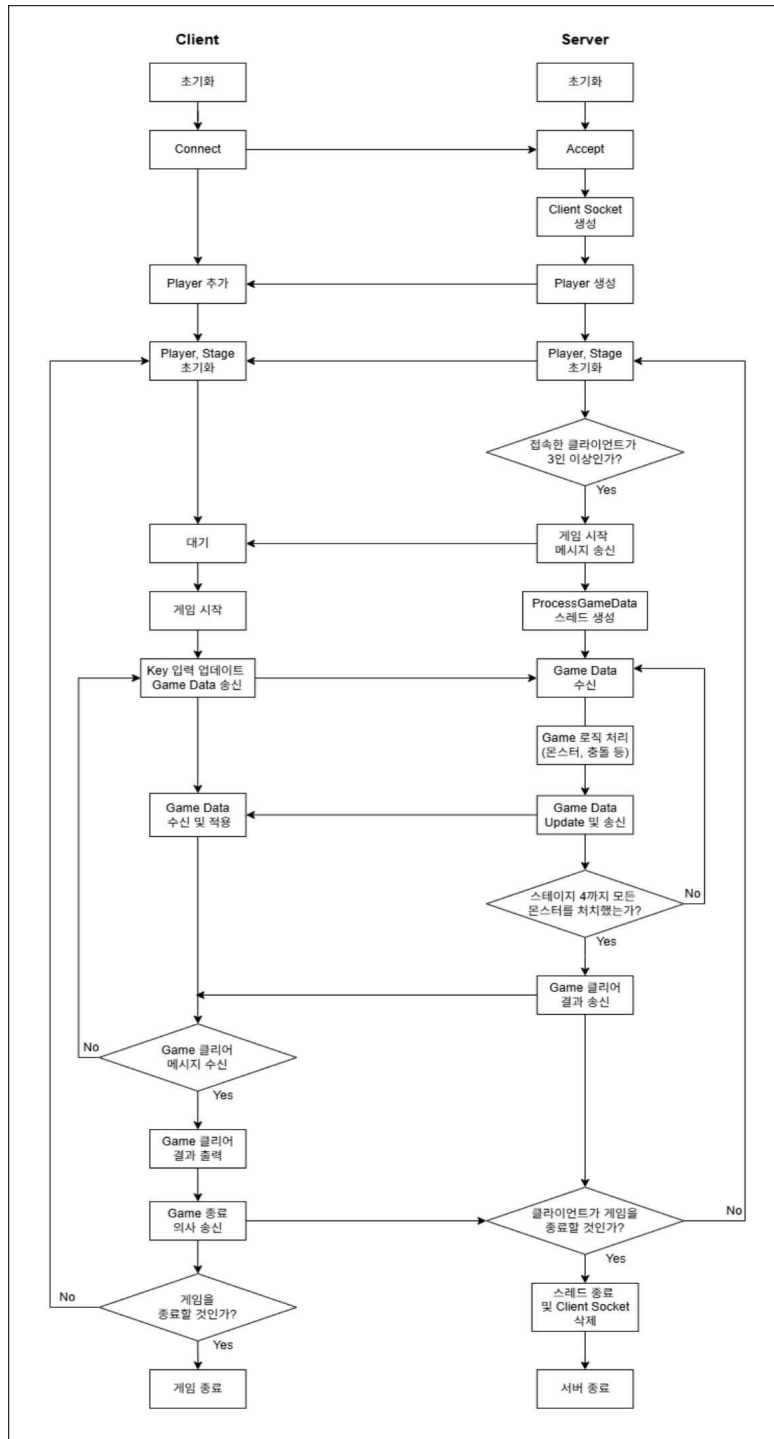


스테이지 4



2. High-level Design

A. 플로우 차트



B. 서버와 클라이언트 흐름

1) 서버와 클라이언트의 연결

[서버]

번호	흐름	설명
1	Winsock 초기화	WSAStartup()을 호출해 소켓 라이브러리 초기화
2	Socket 생성	대기 소켓 생성
3	bind()	서버의 IP 주소와 포트를 socket에 연결
4	listen()	대기 소켓을 생성하고 대기 상태로 전환
5	accept()	클라이언트가 connect 요청하면, 새로운 전용 socket을 생성

[클라이언트]

번호	흐름	설명
1	Winsock 초기화	WSAStartup()을 호출
2	서버 연결	• 서버의 IP 주소와 포트 번호 입력 • connect()를 호출해 서버에 접속 요청을 보냄

2) Game Scene

[서버]

번호	흐름	설명
1	멀티스레드 생성	메인 스레드는 통신용, 새롭게 생성한 스레드는 게임 로직 처리용
2	인원수 체크 후 게임 시작	접속한 클라이언트가 3인 일 때 게임 시작 메시지를 클라이언트에게 보내고 게임 시작
3	플레이어 데이터 수신	• 각 클라이언트의 정보를 수신(Object Type, 위치 등) • 게임 데이터를 업데이트
4	오브젝트 동기화	서버는 클라이언트에게 주기적으로 게임 데이터를 보내 동기화
5	몬스터 AI	몬스터 로직 처리 후 클라이언트에게 송신
6	아이템 및 충돌 처리	아이템 사용, 충돌 확인 등 이벤트 발생 시 데이터를 송신
7	스테이지 클리어	몬스터를 다 처치하면, 다음 스테이지로 전환하라는 메시지를 보냄

[클라이언트]

번호	흐름	설명
1	플레이어 조작	키 입력에 따른 위치, 방향, 상태 등을 서버로 보냄
2	오브젝트 동기화	서버에서 받은 정보로 다른 플레이어 및 오브젝트 동기화
3	스테이지 클리어	스테이지 클리어 여부 수신

3) 결과

[서버]

번호	흐름	설명
1	게임 클리어	모든 스테이지 완료 시 메시지 전송
2	클라이언트 접속 정보 및 스테이지 초기화	게임 종료를 누른 클라이언트의 연결을 종료하고, 다시하기를 누른 클라이언트는 게임 데이터를 초기화함. 게임 스테이지를 초기화함. 게임 데이터 및 게임 스테이지를 초기화함.

[클라이언트]

번호	흐름	설명
1	게임 클리어 수신	클리어 메시지를 받으면, 결과 다시 하기 or 나가기 버튼 출력
2	게임 종료 or 다시하기 선택	버튼을 누르면 서버에 메시지를 보냄. 머무르기 or 나가기

3. Low-level Design

A. 패킷

• 클라이언트 패킷

```
C_UpdateObjectState
C_UpdateObjectState_Packet
{
    int objectID;
    ObjectType type;
    ObjectState state;
}
-
C_UpdateDir
C_UpdateDir_Packet
{
    int objectID;
    ObjectType type;
    Dir dir;
}
C_Move
C_Move_Packet
{
    int objectID;
    ObjectType type;
    Vertex pos;
    Dir dir;
    ObjectState state;
}
C_CreateProjectile_Packet
{
    int objectID;
    ObjectType type;
    Vertex pos;
    Dir dir;
}
C_KickBomb_Packet
{
    Int objectID;
    Dir dir;
}
C_Collision
C_Collision_Packet
{
    CollisionType collisionType;
    int objectID1;
    ObjectType type1;
    Vertex pos1;
    int objectID2;
    ObjectType type2;
    Vertex pos2;
}
C_UseItem
C_UseItem_Packet
{
```

```

        int objectID;
        ObjectType itemType;
        ItemType itemType;
    }

```

```

C_StayGame-
C_StayGame_Packet
{
    int objectID;-
}

```

```

C_EndGame-
C_EndGame_Packet
{
    int objectID;-
}

```

• 서버 패킷

```

S_AddObject
S_AddObject_Packet
{
    int objectID;
    ObjectType type;
    Vertex pos;
    ItemType itemType;
}

```

```

S_RemoveObject
S_RemoveObject_Packet
{
    int objectID;
    ObjectType type;
}

```

```

S_UpdateObjectState
S_UpdateObjectState_Packet
{
    int objectID;
    ObjectType type;
    ObjectState state;
}

```

```

S_UpdateDir
{
    int objectID;
    ObjectType type;
    Dir dir;
}

```

```

S_Move
S_Move_Packet
{
    int objectID;
    ObjectType type;
    Vertex pos;
    Dir dir;
    ObjectState state
}

```

```

-
S_ChangeNextStage
{
    int stageNum;
}

```

```

}

S_CollisionResult
{
    bool result;
}

S_MonsterDamaged
{
    int objectID;
    ObjectType type;
    int monsterHP;
}

S_ItemUseResult
S_ItemUseResult_Packet
{
    bool result;
    ItemType type;
}

S_UpdateTimerPacket
{
    int time
}

S_GetItem_Packet
{
    ItemType type;
}

struct S_EndGame_Packet
{
    bool end;
}

S_SetLife_Packet
{
    int life;
}

```

B. 서버

```

struct Client
{
    Int id;
    SOCKET socket;
    PlayerRef player;
}

template <class T>
struct SendEvtNet
{
    bool isComplete;
    bool isBroadcast;
    SOCKET clientSocket;
    PacketID packetID;
    T packetData;
}

class ServerFramework
{

```

```

public:
    ServerFramework();
    ~ServerFramework();

public:
    void Update();
    void ProcessRecv(ClientRef client);

    template <class T>
    void ProcessSend(PacketID id, const T& packetData, SOCKET clientSocket);

    template <class T>
    std::vector<char*> CreatePacket(PacketID id, const T& packetData);

public:
    void SendAddObjectPacket(GameObjectRef object, bool broadcast, SOCKET client = 0);
    void SendMovePacket(GameObjectRef object, bool broadcast, SOCKET client = 0);
    void SendUpdateTimerPacket(GameObjectRef object, bool broadcast, SOCKET client = 0);
    void SendUpdateTimerPacket(bool broadcast, SOCKET client = 0);
    void SendGetItemPacket(ItemRef item, PlayerRef player);
    void SendItemUseResultPacket(bool result, ItemType type, bool broadcast, SOCKET client
= 0);
    void SendEndGamePacket(bool broadcast, SOCKET client = 0);
    void SendSetLifePacket(PlayerRef player);

    template <class T>
    void Broadcast(PacketID id, const T& packetData);

private:
    void ProcessAccept(SOCKET clientSocket);
    void ProcessDisconnect(ClientRef client);
    void ProcessMovePacket(C_Move_Packet packet);
    void ProcessCreateProjectilePacket(C_CreateProjectile_Packet packet);
    void ProcessUseItemPacket(C_UseItem_Packet packet);
    void ProcessKickBombPacket(C_KickBomb_Packet packet);

public:
    Room* GetRoom();

    void AddRemoveObject(GameObjectRef object);
};

class Room // 게임 내 Object를 한 번에 관리하는 클래스
{
public:
    Room();
    ~Room();

public:
    void Update();
    void SpawnMonster();

private:
void AddObject();
    GameObjectRef AddObject(ObjectType type, Vertex pos = {-1, -1}, Dir dir = Dir::None);
    void RemoveObject(ObjectType type, int id);
void InitPlayer();
void StartGame();

    void ChangeStage();
    void ClearStage();
    void EndGame();

```

```

public:
    std::unordered_map<int, PlayerRef>& GetPlayers();
    std::unordered_map<int, MonsterRef>& GetMonsters();
    std::unordered_map<int, ProjectileRef>& GetProjectiles();
    std::unordered_map<int, BombRef>& GetBombs();
    GameObjectRef GetObject(ObjectType type, int id);
    RoomState GetRoomState();
    void SetRoomState(RoomState state);
    int GetTimer();
};

class State {
public:
    virtual void Enter() = 0;
    virtual void Exit() = 0;
    virtual void Tick() = 0;
};

class Player : public GameObject {
public:
    Player();
    virtual void Update();
    bool IsCaneShoot();
}

class IdleState {
public:
    virtual void Enter(GameObject* object) override;
    virtual void Exit(GameObject* object) override;
    virtual void Doing(GameObject* object) override;
};

class MoveState : public State {
public:
    virtual void Enter() override;
    virtual void Exit() override;
    virtual void Tick() override;
};

class MoveToTargetState : public State {
public:
    virtual void Enter() override;
    virtual void Exit() override;
    virtual void Tick() override;
};

class SetTargetState : public State {
public:
    virtual void Enter() override;
    virtual void Exit() override;
    virtual void Tick() override;
};

class BombState : public State {
public:
    virtual void Enter(GameObject* object) override;
    virtual void Exit(GameObject* object) override;
    virtual void Tick(GameObject* object) override;
};

class DeadState : public State {
public:
    virtual void Enter() override;

```

```

        virtual void Exit() override;
        virtual void Tick() override;
};

class UseItemState : public State {
public:
    virtual void Enter() override;
    virtual void Exit() override;
    virtual void Tick() override;
};

class UseSkillState : public State {
public:
    virtual void Enter() override;
    virtual void Exit() override;
    virtual void Tick() override;
};

class StateMachine {
public:
    StateMachine(GameObject* object);
    ~StateMachine();
    void Start();
    void Update();

    void ChangeState(State* state);
private:
    GameObject* _object{};
    State* _curState{};
};

enum class ObjectType
{
    Player,
    Monster,
    Button,
    Item,
    Bullet,
};

struct Vertex
{
    int x, y;
};

struct Status {
    int _hp, _speed;
};

class GameObject {
public:
    GameObject();
    ~GameObject();
    virtual void Update();
    virtual void Move();
    virtual void FindTarget();

    void SetObjectType(ObjectType type);
    ObjectType GetObjectType();
    void SetPos(Vertex pos);

    StateMachine* GetStateMachine();
    Vertex GetTargetPos();
    void SetTargetPos(Vertex target);
    Vertex GetPos();
};

```

```

}

class BombObject : public GameObject {
public:
    BombObject();
    void Update() override;
};

class Monster : public GameObject {
public:
    Monster();
    void Move() override;
    void Update() override;
    void UseSkill();
};

class TankMonster : public Monster {
public:
    TankMonster();
    void FindTarget() override;
};

class RespawnMonster : public Monster {
public:
    RespawnMonster();
    void FindTarget() override;
};

class ObstacleMonster : public Monster {
public:
    ObstacleMonster();
    void FindTarget() override;
};

class NormalMonster : public Monster {
public:
    NormalMonster();
    void FindTarget() override;
};

class BomberMonster : public Monster {
public:
    BomberMonster();
    void FindTarget() override;
};

enum class ItemType
{
    Life,
    Magazine,
    Lightning,
    Waterwheel,
    Coffee,
    Shotgun,
    Hourglass,
};

class Item : public GameObject {
public:
    Item(ItemType);
    virtual void Update();
    virtual void ChangeState();
    void Expired();
};

```

```

class Bomb : public GameObject {
public:
    virtual void Update();
};

class Projectile : public GameObject {
public:
    virtual void Update();
    virtual void Move();
};

```

C. 클라이언트

```

class GameNetwork
{
public:
    GameNetwork();
    ~GameNetwork();

public:
    void Update();

private:
    void ProcessSend();
    template<class T>
    void ProcessSend(PacketID id, const T& packet);

    void ProcessRecv();

    // Send Packet
    void SendUpdateObjectStatePacket();
    void SendUpdateDirPacket();
    void SendCollisionPacket();
    void SendStayGamePacket();
    void SendEndGamePacket();
    void SendMovePacket();
    void SendUseItemPacket();
    void SendKickBombPacket();
    void SendUseItemPacket();

    // Recv 동작
    void RecvAddObject();
    void RecvRemoveObject();
    void RecvUpdateObjectState();
    void RecvUpdateDir();
    void RecvMove();
    void RecvChangeNextStage();
    void RecvCollisionResult();
    void RecvMonsterDamaged();
    void RecvItemUseResult();
    void RecvUpdateTimer();
    void RecvGetItem();
    void RecvEndGame();
    void RecvSetLife();

public:
    template <class T>
    std::vector<char*> CreatePacket();

    void SetGameScene();
}

```



```

private:
    SOCKET _socket;

    fd_set _readSet{};
    fd_set _writeSet{};

    GameScene* _gameScene{};
};

class GameObject
class GameScene
{
public:
    void SyncData(); // 변경된 정보를 GameNetwork에 알림
    void SyncPlayer();
    void SyncMonster();
    void SyncObject();

    void GetItemLocalPlayer();
    void SyncObjectTimer();

    void SetPlayerState();
    void SetMonsterState();

    void SetLifeOfLocalPlayer();

    void SetObjectState();

    virtual void ProcessInput() override;
private:
    PlayerRef _localPlayer;

    std::unordered_map<int, PlayerRef> _players;
    std::unordered_map<int, MonsterRef> _monsters;
    std::unordered_map<int, GameObjectRef> _objects;
    std::vector<GameObjectRef> _obstacles;

    GameNetwork* _gameNetwork{};
};

```

D. 멀티 스레드 함수

```
// 서버
DWORD WINAPI ProcessGameData();

// 클라
DWORD WINAPI ProcessNetwork();
```

공유 자원	서버: Room class가 관리하는 GameObject vector unordered_map, Server Framework가 관리하는 SendEvent vector들 클라이언트: GameScene이 관리하는 GameObject unordered_map들
멀티 스레드 동기화 기법	Event 임계 영역
생성하는 스레드	서버: 클라이언트가 접속할 때마다 Main 실행시 ProcessGameData 스레드를 생성 클라이언트: connect 후 recv 스레드 생성
SetEvent() 호출 임계 영역	vector원소 삭제/추가 시 공유자원 접근 및 수정시

	클라이언트	서버
스레드 종류	• Main Thread: 로직처리, 렌더, send • Network Thread: recv	• Main Thread: 통신 전용 • Process Game Data: 로직 처리
동기화 기법	임계 영역	임계 영역

4. 팀원 간 역할 분담

임채은	클라이언트/서버 연동을 위한 클라이언트 코드 추가 <pre> GameNetwork::GameNetwork() GameNetwork::~GameNetwork() GameNetwork::Update() GameNetwork::ProcessSend() GameNetwork::SendMovePacket() GameNetwork::SendKickBombPacket() GameNetwork::ProcessRecv() GameNetwork::RecvAddObject() GameNetwork::RecvRemoveObject() GameNetwork::RecvUpdateDir() GameNetwork::RecvMove() GameNetwork::RecvChangeNextStage() GameNetwork::RecvCollisionResult() GameNetwork::RecvMonsterDamaged() GameNetwork::RecvItemUseResult() GameNetwork::UpdateTimer() GameObject::SyncData GameScene::SyncPlayer() GameScene::SyncMonster() GameScene::SyncObject() </pre>
최미나	멀티 스레드 동기화 <pre> Monster::UseSkill() TankMonter::FindTarget() NormalMonter:: FindTarget() ObstacleMonter:: FindTarget() RespawnMonter:: FindTarget() BomberMonter:: FindTarget() MoveState:: Enter, Exit, Tick MoveToTargetState::Enter, Exit, Tick SetTargetState:: Enter, Exit, Tick BombState:: Enter, Exit, Tick DeadState:: Enter, Exit, Tick UseItemState:: Enter, Exit, Tick UseSkillState:: Enter, Exit, Tick StateMachine::StateMachine(GameObject* object); StateMachine::~StateMachine(); StateMachine::Start(); StateMachine::Update(); </pre>

	<pre> StateMachine::ChangeState(State* state); GameObject::GameObject(); GameObject::Move(); GameObject::FindTarget(); Player::Player(); Player::Update(); DWORD WINAPI ProcessGameData() Item::Item(ItemType) Item::Update(); Item::ChangeState(); Item::Expired(); Bomb::Update() Projectile::Update() Projectile::Move() Room::Update() // 몬스터 로직 구현 몬스터 이동 동기화 아이템 획득/사용 동기화 S_SetLife_Packet 동기화 </pre>
임수영	• 서버/클라 연동 및 ServerFramework, Room 구현 <pre> ServerFramework::ServerFramework() ServerFramework::~ServerFramework() ServerFramework::Update() ServerFramework::ProcessSend() ServerFramework::ProcessRecv() ServerFramework::CreatePacket() ServerFramework::SendAddObjectPacket() ServerFramework::SendRemoveObjectPacket() ServerFramework::SendUpdateObjectStatePacket() ServerFramework::SendMovePakcet() ServerFramework::UpdateTimerPacket() ServerFramework::GetItemPacket() ServerFramework::ItemUseResultPacket() ServerFramework::EndGamePakcet() ServerFramework::Broadcast() ServerFramework::ProcessAccept() ServerFramework::ProcessDisconnect() ServerFramework::ProcessMovePacket() ServerFramework::ProcessCreateProjectilePacket() ServerFramework::ProcessUseItemPacket() ServerFramework::ProcessKickBombPacket() ServerFramework::GetRoom() </pre>

ServerFramework::AddRemoveObject()

Room::Room()
Room::~~Room()
Room::Update()
Room::AddObject()
Room::RemoveObject()
~~**Room::InitPlayer()**~~
~~**Room::StartGame()**~~
Room::EndGame()
~~**Room::ChangeStage()**~~
Room::ChangeStage()
Room::ClearStage()
Room::EndGame()
Room::GetPlayers()
Room::GetProjectiles()
Room::GetBombs()
Room::GetObject()
Room::GetRoomState()
Room::SetRoomState()
Room::GetTimer()

GameObject::SetDir()
GameObject::GetDir()
GameObject::GetState()

Projectile::Move()
Projectile::Update()

ProcessGameData()

Server 멀티쓰레드 동기화

5. 개발 환경

작업 IDE	Visual Studio 2022
버전 관리	Github
개발 언어	C++
네트워크 프로토콜 및 API	TCP/IP, Winsock
입출력(I/O) 모델	Select
코딩 컨벤션	- 변수 이름: camelCase, 멤버 변수 앞에 _ 붙이기 - 함수 이름: PascalCase - 클래스 이름: PascalCase

6. 개발 일정(11월~12월) ■: 임수영 ■: 최미나 ■: 임채은

* 수정된 일, 없앤 일 짝짝 * 추가된 일 파란색 글씨 * 완료된 일 노란 형광펜

월	화	수	목	금	토	일
					1	2
						클라/서버 연동
3	4	5	6	7	8	9
	[ServerFramework] ServerFramework(), ~ServerFramework(), Update() [Room] Room(), ~Room()					[ServerFramework] Update(), ProcessSend(), ProcessRecv(), GetRoom(), CreatePacket(), ProcessGameData()
1	11	12	13	14	15	16

0						
	[Room] AddObject() [ServerFramework] Update(), ProcessRecv() }, ProcessSend() CreatePacket()					
17	18	19	20	21	22	23
			[ServerFramework] AddObject() [Room] GetObjects(), 멀티스레드 동기화	[ServerFramework] ProcessMovePacket(), ProcessAccept(), Broadcast(), AddObject(), Update()	[ServerFramework] Update(), ProcessRecv(), ProcessSend(), ProcessAccept(), ProcessDisconnect(), ProcessMovePacket(), RemoveObject() [Room] RemoveObject()	[Room] StartGame(), ChangeStage() [ServerFramework] ProcessRecv(), ProcessSend(), ProcessMovePacket()
24	25	26	27	28	29	30
	[Room] EndGame(), Update(), SetRoomState(), AddObject(), RemoveObject() [ServerFramework] ProcessSend(), ProcessRecv(), ProcessMove		[ServerFramework] SendUpdateTimerPacket(), SendMovePacket(), ProcessAccept() [Room] GetTimer(), Update()		[ServerFramework] ProcessRecv(), ProcessSend(), ProcessCreateProjectilePacket() [Room] AddObject()	[ServerFramework] ProcessRecv(), ProcessSend(), Update(), SendAddObjectPacket(), SendRemoveObjectPacket(), SendMovePacket(), AddRemoveObject(), ProcessAccept() [Projectile] Move(), Update()

	ePacket(), ProcessAccept() [GameObject] SetDir(), GetDir(), GetState() ProcessGameData()					
1	2	3	4	5	6	7
	[Room] AddObject(), RemoveObject(), GetObject(), GetPlayers(), Update() [ServerFramework] Update(), AddRemoveObject()	[ServerFramework] SendUpdateObjectStatePacket(), SendGetItemPacket() [Room] ChangeStage(), ClearStage()		[ServerFramework] ProcessUseItemPacket(), SendItemUseResultPacket(), ProcessMovePacket() [Player] GetItem()	테스트 및 버그 수정 [Room] ChangeStage(), Update()	테스트 및 버그 수정 [ServerFramework] SendEndGamePacket(), Update(), SendItemUseResultPacket(), ProcessProcessKickBombPacket() [Room] Update(), GetBombs()
8	9					
	최종 테스트 [Room] GetProjectiles()					

원	화	수	목	금	토	일
					1	2
					TankMonter, NormalMonter, ObstacleMonter, RespawnMonter, BomberMonter 클래스의 FindTarget(),	[GameObject] GameObject(), Move(), FindTarget(), [Projectile] Update(), Move()
3	4	5	6	7	8	9
	MoveToTargetState, SetTargetState, BombState, DeadState, MoveState UseItem class UseSkillState, UseSkill()		[Player] Player() Update()	[Player] Player() Update()	[Player] Update()	{Room} Update();
10	11	12	13	14	15	16
[StateMachine] StateMachine(GameObject* object), ~StateMachine(), Start(), Update(), ChangeState(State* state)	[Item] Item(ItemType), Update() ChangeState() Expired()		[Player] Player() Update() 수정			
17	18	19	20	21	22	23
		[Room] Update();			ProcessGameData()	ProcessGameData()
24	25	26	27	28	29	30
ProcessGameData()	ProcessGameData()		몬스터 이동 동기화 (버그 발생)		ProcessGameData() 동기화	ProcessGameData() 동기화 버그 수정
1	2	3	4	5	6	7
		[Room] Update(); 몬스터 충돌 구		[GameNetwork] GetItem_Packet recv 구현	테스트 및 버그 수정	테스트 및 버그 수정 S_SetLife_Packet 동기화 아이템 사용/획득 동기화

		연				
8	9	10				
테스트 및 버그 수정	최종 테스트					

일	화	수	목	금	토	일
					1	2
					{GameNetwork} GameNetwork() — connect 연결 요 청 ~GameNetwork(), Update()	[GameNetwo rk] GameNetwo rk() - connect 연결 요청 ~GameNetwo rk(), Update()
3	4	5	6	7	8	9
	[GameNetwork] ProcessSend(), Update()				[GameNetwork] ProcessSend(), Update()	[GameNetwo rk] CreatePacket(), ProcessSend(), Select 모델 로 변경
10	11	12	13	14	15	16
	{GameNetwork} ProcessSend(), Update()	[GameNetwo rk] ProcessRecv () - 데이터 추출		[GameScen e] AddPlayer(), AddMonste r()	{GameNetwork} ProcessSend() — Update() [GameScene] LocalPalyer 추가	
17	18	19	20	21	22	23
	GameObject::SyncData() [GameNetwork] ProcessRecv(), Send() - SendUpdateObjectState Packet(), SendUpdateDirPacket(), SendMovePacket(), SendCollisionPacket(), SendUseItemPacket(), SendStayGamePacket(), SendEndGamePacket()			[GameNetwo rk] C_Move_Pa cket Send 테스트	GameObject::Sync Data() [GameNetwork] ProcessRecv() - S_Move_Packet, S_AddObject_Pack et, S_RemoveObject_ Packet Recv 후 로 직 구현 ~Send() [GameScene]	[GameScene] Player 키보 드 입력시 C_Move_Pack et 전송

					플레이어, 몬스터, 객체 vector -> unordered_map 변경	
24	25	26	27	28	29	30
[GameScene] SyncPlayerPos() (localPlayer만 움직이는 문제 해결)	GameObject::SyncData() GameScene::SyncPlayer() [GameNetwork] ProcessRecv(), Send() - Move 패킷 수정, 이동 동기화 시 방향과 State 동기화 처리		[GameScene] _localPlayer와 _players 나눔, 몬스터 Move 패킷 받는 로직 수정	[GameNetwork] Send - 총알 쏘기	GameObject::SyncData() [GameNetwork] ProcessRecv(), Send() - 총알 위치 sync	
1	2	3	4	5	6	7
	[GameNetwork] Recv용 멀티스레드 나눔	[GameNetwork] Select 제거, 멀티스레드 수정	총알 렉걸리는 문제 해결		테스트 및 버그 수정 UI(ProgressBar) 수정, Title Scene 추가(IP 주소 입력 컨트롤), S_EndGame 패킷 추가	테스트 및 버그 수정 KickBomb 패킷 추가, 폭탄 발로 차기 send, 로컬 플레이어 죽으면 안 그리기
8	9	10				
테스트 및 버그 수정	최종 테스트					

7. 오류 및 해결 과정

번호	오류 내용	발생 원인	해결 과정
1	클라이언트 2명 접속시 서버가 터짐	서버 Room 클래스의 _objects 공유 자원 접근 문제 1. ServerFramework의 Update()에서 매번 Room::Update()를 호출하며 _objects를 읽음 2. 클라이언트가 접속할 때 Room::AddObject()를 호출하여 _objects에 값을 씀	_objects에 접근시 임계영역 사용
2	클라이언트가 여러 명 접속했을 때, 첫 번째 클라이언트의 접속을 종료하면 서버가 터짐	for문을 돌 때 바로 _clients에 접근	_removeClients로 삭제 예약 후 나중에 삭제
3	ServerFramework에서 object 관리하는 자료형을 vector에서 unordered_map으로 변경했더니, 첫 번째 클라의 플레이어가 안 움직임(두 번째 클라는 첫 번째 클라의 움직임이 그려짐)	서버 문제가 아니라 클라이언트 문제였음. GameScene에서 관리하는 _localPlayer와 _players의 동기화가 안 됐던 것	_localPlayer와 _players를 완전히 분리함. 기존에는 로컬 플레이어도 _players에 넣었는데, 이젠 다른 플레이어만 관리하도록 수정함.
4	플레이어와 몬스터가 동시에 이동하면 클라이언트에 패킷이 보내지지 않는 버그 발생	패킷을 동시에 send해서 송신버퍼에 값이 섞여서 저장. 클라이언트에 패킷 사이즈가 쓰레기 값으로 recv	바로 send하지 않고 컨테이너를 만들어서 한 번에 send 처리
5	서버가 갑자기 터짐, 아이템 GetID가 nullptr이라는 액세스 위반	_removeObjects 가 다른 곳에서 수정	임계영역 사용